Module 5: Part 2

Server Technology

Server technology refers to the hardware and software systems that provide services, resources, or data to client systems over a network. In a client–server architecture, servers play a central role by handling client requests, processing data, and ensuring reliable communication.

Key Features of Server Technology

Centralized Resource Management: Servers manage shared resources such as files, databases, and applications.
Concurrency: Capable of handling multiple client requests simultaneously using multithreading or multiprocessing.
Scalability: Supports vertical (hardware upgrade) and horizontal (adding more servers) scaling.
Reliability and Availability: Uses redundancy, backup, and fault-tolerant mechanisms to ensure continuous service.
Security: Implements authentication, authorization, encryption, and access control.

Types of Servers

File Server – Manages file storage and access.
Database Server – Handles database queries and transactions.
Web Server – Serves web pages using HTTP/HTTPS.
Application Server – Hosts business logic and application services.
Mail Server – Manages email communication

Balanced Computing

Balanced computing refers to a computing approach where **processing responsibility is distributed appropriately between the client and the server**, instead of placing all computation on either side.

In earlier systems:

- **Mainframe era** → Server did *everything*
- **Early client/server** → Server still dominant, client mostly UI

Balanced computing emerged when designers realized that:

- Clients had become powerful (PCs, browsers, rich GUIs)

- Servers should focus on **shared services, coordination, and data integrity**
- Overall system performance improves when **workload is balanced**, not centralized

Characteristics of Balanced Computing

- Clients handle:
    - Presentation logic (UI, validation)
    - Some business logic
- Servers handle:
    - Data management
    - Security
    - Transaction control
    - Shared business rules
- Network traffic is optimized
- Scalability improves

Example (Web Application):

- Browser validates form input (client-side)
- Server processes payment and database updates (server-side)

Server's Changing Role

The **role of the server has evolved** significantly as computing architectures matured.

Earlier Role of Servers

- Centralized processing
- Heavy computation
- Dumb clients (terminals)
- Monolithic systems

Changing Role in Modern Client/Server & Web Systems

Servers are no longer just “compute machines”. They now act as:

- **Service providers**
- **Data managers**
- **Security enforcers**
- **Transaction coordinators**
- **Integration hubs**

Instead of doing *everything*, servers:

- Expose **services/APIs**
- Support **multiple clients simultaneously**
- Coordinate distributed systems

In Web Applications

- Servers respond to HTTP/API requests
- Servers generate or assemble dynamic content
- Servers interact with databases and other services
- Servers enforce authentication and authorization

Thin client computing-computing models

A thin client is a basic computing device that runs services and software from a centralized server. With a thin client, only a monitor, network card, mouse, and keyboard are available to the end user, and other hardware and services run on the network.Thin clients function as access points to a centralized server. They rely heavily on the server for computational tasks, primarily displaying processed data and sending user inputs back to the server.

Here are the fundamentals behind how thin clients work:

- **Centralized server**: A centralized server is at the core of the thin client model. This server runs applications, processes data, and stores files. It’s a powerful system designed to handle multiple user requests simultaneously.
- **Network connectivity**: Thin clients connect to this centralized server via a network, usually through a local area network (LAN) or over the internet. The quality and speed of this connection are crucial since they affect the user experience.
- **Minimal local processing**: Unlike traditional PCs, thin clients perform minimal processing on the device. They primarily display information processed on the server

and send user input (like keyboard strokes and mouse movements) back to the server.

- **Standard protocols**: Thin clients communicate with the server using standard protocols. Common ones include ICA (Independent Computing Architecture) from Citrix or RDP (Remote Desktop Protocol) from Microsoft. These protocols ensure efficient and secure communication between the client and the server.
- **Stateless operation**: Most thin clients are stateless, meaning they don't store user settings or data locally. When a user logs off, the device returns to its default state, making it secure and easy to manage since there's no user data to manage or protect on the device itself.
- **Remote desktop environment**: When users interact with a thin client, they access a remote desktop environment. The server provides each user with a distinct session that feels like a typical desktop experience, even though the actual processing is happening on the server.
- **Updates and maintenance**: Since the primary software and data are stored on the central server, updates, patches, and maintenance are centralized. So, when IT teams update the server, it applies to all connected thin clients, simplifying management and ensuring consistency across all devices.

This functionality is advantageous for security, management, and cost but also relies on stable network connectivity and a robust central server for optimal performance.

Comparison-computing environment

The computing environment is defined by a division of labor between dedicated clients that request services and a centralized server that provides them. This contrasts with other models, such as peer-to-peer, where any node can be both a client and a server.

In thin-client computing, different models exist based on how processing and data are shared between the client and the server. These models operate in different computing environments and provide different levels of performance, cost, security, and flexibility.

The main thin-client computing models are the Host-based model, Client–Server model, Web-based thin client model, and Hybrid client model.

1. **Host-Based vs Client–Server Model**

   In the host-based model, all application processing takes place on a central server. The client only sends user input and displays output. The computing environment is completely centralized, which makes data highly secure and easy to manage. However, the server carries a heavy workload, and if the server fails, the entire system stops working.

   In contrast, in the client–server thin client model, some processing such as user interface handling is done on the client, while business logic and data storage remain on the server. This creates a distributed computing environment. The server load is reduced compared to the host-based model, and the system performs better, but still maintains centralized data control.

2. **Web-Based Thin Client vs Traditional Thin Client**

In traditional thin-client systems, special client software is required to connect to the server. In a web-based thin-client model, the web browser itself acts as the client. The server hosts the application and sends web pages to the browser.

The computing environment in web-based thin clients is platform-independent and internet-based. Users can access the system from anywhere using any device with a browser. Compared to traditional thin clients, web-based environments are easier to deploy and maintain.

3. **Hybrid Client vs Pure Thin Client**

   A pure thin client depends almost completely on the server for processing and storage. This provides high security and easy management but can suffer from network delays.

   A hybrid client, however, performs some processing locally while still relying on the server for data and important services. Its computing environment balances performance and control. Hybrid clients provide better responsiveness than pure thin clients and reduce server load, but they require more powerful client devices.

Overall Comparison of Computing Environments

Host-based environments are fully centralized, secure, and easy to manage but require powerful servers.

Client–server thin-client environments balance server and client processing, improving performance.

Web-based environments provide global access, easy updates, and platform independence.

Hybrid environments offer flexibility by combining local processing with centralized data control.

Thus, thin-client computing environments differ mainly in how much responsibility is given to the client and the server, and organizations choose the model based on cost, performance, scalability, and security requirements.

Future of Client/Server Enabling Technologies

Client/server architecture continues to evolve with advancements in networking, cloud computing, and intelligent systems. The future of client/server enabling technologies focuses on improving scalability, performance, security, and flexibility.

One major development is the **shift towards cloud-based client/server models**, where servers are hosted on cloud platforms. This enables dynamic resource allocation, high availability, and cost efficiency. Clients can access services from anywhere using lightweight applications or web browsers.

Another important trend is the **adoption of microservices architecture**, where server-side applications are broken into smaller, independent services. This improves maintainability, fault tolerance, and faster deployment compared to traditional monolithic servers.

Edge computing is also shaping the future by moving computation closer to the client. This reduces latency and improves real-time processing, which is essential for IoT, autonomous systems, and multimedia applications.

The integration of AI and machine learning into client/server systems enables intelligent decision-making, predictive analytics, and personalized services. Servers can analyze client data to optimize performance and user experience.

Security technologies such as **zero-trust architecture**, **encryption**, and **secure authentication** mechanisms are becoming essential to protect distributed client/server environments from cyber threats.

Overall, the future of client/server enabling technologies lies in cloud-native, intelligent, secure, and highly scalable systems that can support modern applications and global connectivity.

TRANSFORMATIONAL SYSTEMS

Transformational systems refer to technologies enabled by **Client/Server computing** that have **fundamentally changed how organizations operate, communicate, and conduct business processes**.

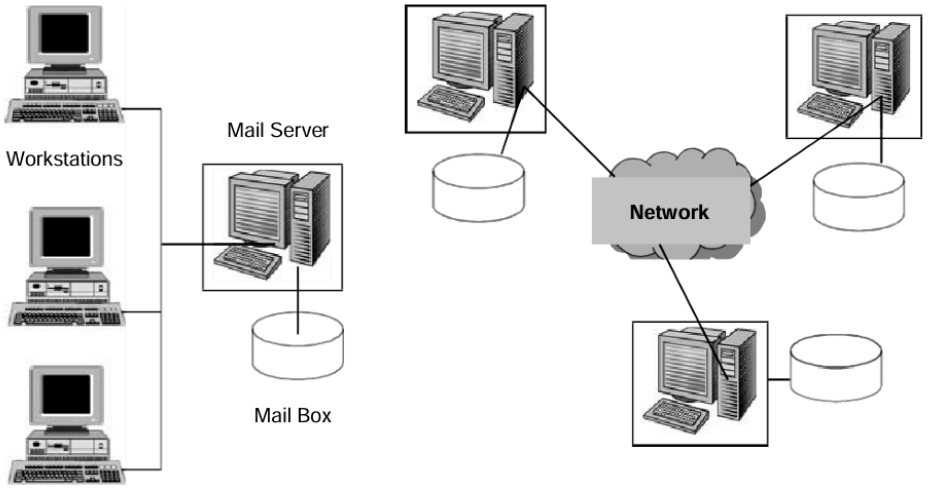
Examples:

1. Email
2. Client/Server and user security
3. Object Oriented Technology - CORBA
4. Electronic Data Interchange

1. Electronic Mail (E-mail)

E-mail is the most heavily used network application in the corporate world, allowing users to compose and exchange messages.

- Traditional e-mail was limited and inflexible.
- **Intranet Mail:** Provides standards and simple methods for attaching documents, sound, images, and multimedia. It can manage mailing lists that act as aliases for multiple destinations.
- **Single System:** Users on a shared system exchange messages by placing them in the recipient's "mailbox," which is essentially a file directory managed by the file management system.
- **Distributed System:** Uses mail servers (handlers) connected over a network (WAN or Internet) to exchange mail between different systems.



(a) Single System (b) Networks of Mail System

Downloaded from Kaustav.in

2. Client/Server and User Security

The distribution of services in Client/Server models makes them vulnerable to security breaches, fraud, and misuse. Security must consider hosts, PCs, LAN, WAN and users

- **Desktops (Front-end):** The least secure environment as it directly deals with user input. Open connections can provide entry points for hackers.
- **Physical Security:** Risks include unauthorized loading of software/viruses and access to configuration files. **Unauthorized loading of software or viruses**, either through removable media or direct installation. **Direct access to configuration and system files**, since most desktop operating systems assume that the person who turns on the system is the legitimate owner.
- **User Behavior:** Users are often the primary security weakness (e.g., sharing passwords, picking easy-to-guess passwords).
- **Authentication:** User identification and authentication are the front-line defense. Authentication ensures that only legitimate users can access systems and services. Without proper authentication, distributed client/server systems become highly vulnerable due to open network connections. Authentication protects not only applications but also databases and server resources accessed through the client.
- **Password Management:** Enforcing a **minimum number of characters** in passwords.
  Preventing easily guessable passwords.
  Requiring **regular password changes**.
  Encouraging **passphrases instead of simple passwords**, as they are harder to crack.
- **Gateway Security:** Securing network gateways prevents malware from reaching clients.

EXAMPLE: Traditional phone-based dispatch systems are replaced by **GUI-based client/server systems**, reducing operational cost and improving efficiency.
AVL and digital map tracking enable real-time vehicle location, obstacle awareness, and dynamic routing, increasing reliability and response speed.
The system is **fault-tolerant**—if one dispatcher workstation fails, another client can continue operations using the same server data.

3. Object-Oriented Technology: CORBA

CORBA (Common Object Request Broker Architecture) combines object technology with the Client/Server model. It ensures that everything on the network is viewed as an object.

Object Request Broker (ORB):

- Acts as a directory of remote services.
- Routes requests between clients and server objects, managing identity and location transparency.

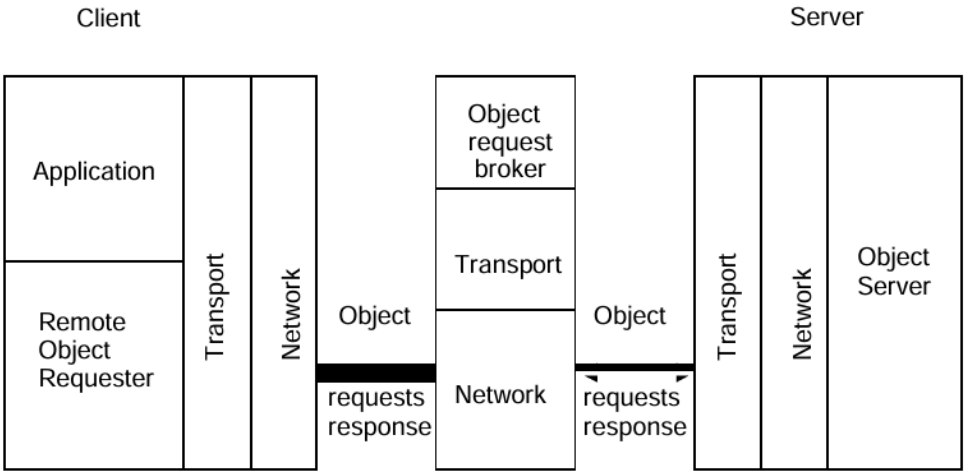


Fig. 9.5: Object Request Broker

Overview of CORBA

CORBA is an **object-oriented, distributed computing framework** defined by the **Object Management Group (OMG)** in 1989. It provides a **standard architecture** that allows software components (objects) running on **different machines, operating systems, networks, and programming languages** to communicate seamlessly.

The key idea of CORBA is that **everything on the network is treated as an object**, and clients can invoke services on remote objects **without knowing their physical location or implementation details**. This makes CORBA highly suitable for **heterogeneous enterprise environments** where integration and reuse are essential

CORBA – Introduction

CORBA is a specification for **Distributed Object Management (DOM)**. It combines:

- **Object-Oriented Technology**
- **Client/Server Architecture**
- **Open Distributed Computing**

In CORBA, when a client needs a service, it does not communicate directly with the server. Instead, it sends a request to an **Object Request Broker (ORB)**, which acts as an intermediary or directory of services.

The ORB:

- Locates the required remote object
- Transfers the request to that object
- Receives the response
- Returns the response back to the client

This mechanism hides network complexity and enables **location transparency, platform independence, and language neutrality**

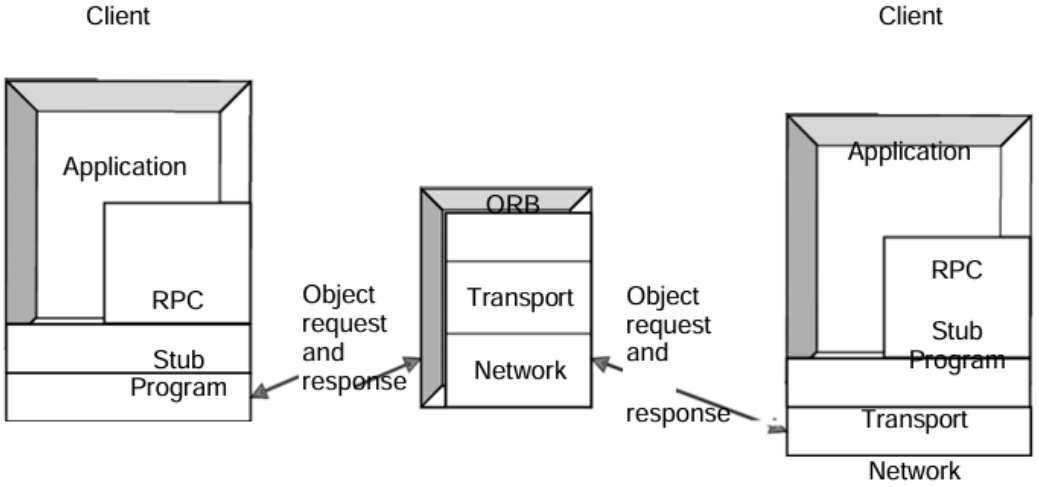


Fig. 9.6: CORBA Remote Services

CORBA Architecture and Object Request Broker (ORB)

The **Object Request Broker (ORB)** is the core component of CORBA.

Role of ORB:

- Routes requests between clients and servers
- Manages object identity and location
- Handles communication and data transfer
- Ensures that method calls appear local to the client

Clients send requests to the ORB, which forwards them to the appropriate **remote object**. The server processes the request and sends the response back through the ORB.

ORB also supports:

- Directory services
- Messaging
- Security
- Location transparency
- Error handling

Thus, ORB significantly **reduces complexity** in distributed client/server interactions

CORBA Concepts

CORBA is based on **three fundamental concepts**:

(i) Object-Oriented Model

CORBA follows a pure object-oriented approach where:

- **Objects** are encapsulated entities providing services
- **Clients send requests** to objects
- Requests include:
    - The operation to be performed
    - Optional parameters

Key object model elements:

- **Object**: Provides one or more services
- **Request**: Action initiated by a client
- **Interface**: Defines operations an object supports
- **Operation**: Specific service offered by an object
- **IDL (Interface Definition Language)**: Used to define standard, platform-independent interfaces

Object creation and destruction are managed based on system requirements

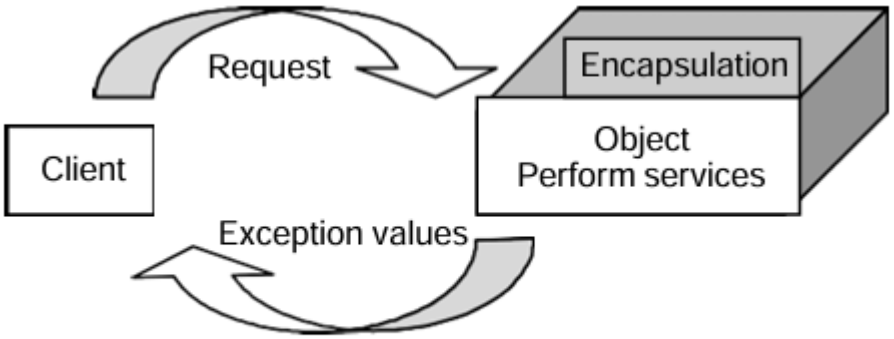


Fig. 9.8: Object Implementation

.

(ii) Open Distributed Computing Environment

CORBA extends the traditional client/server model by **introducing a broker** between client and server.

This broker:

- Insulates applications from hardware, OS, network protocol, and programming language differences
- Provides common services such as:
    - Communication
    - Naming and directory services
    - Security
    - Metadata description
    - Exception handling

As a result, applications can interact across **heterogeneous platforms** without modification

.

(iii) Component Integration and Reuse

CORBA enables:

- Integration of existing components
- Reuse of software modules
- Standardized interfaces defined once using IDL

Once interfaces are defined, the ORB handles all future interactions. This supports **rapid application development**, scalability, and long-term maintainability in enterprise systems

.

CORBA Client and Server

In CORBA, the distinction between client and server is **context-dependent**.

CORBA Server

- A component is considered a server if it **creates and hosts CORBA objects**
- It provides implementations for object methods
- Executes methods on behalf of clients

CORBA Client

- A component is considered a client if it **invokes methods** on remote CORBA objects
- It accesses services through object references obtained via the ORB

Important Point

A single CORBA component can act as **both a client and a server simultaneously**:

- It may provide services to other components
- While also requesting services from other objects

Thus, CORBA supports **bi-directional, flexible interactions** unlike rigid traditional client/server models.

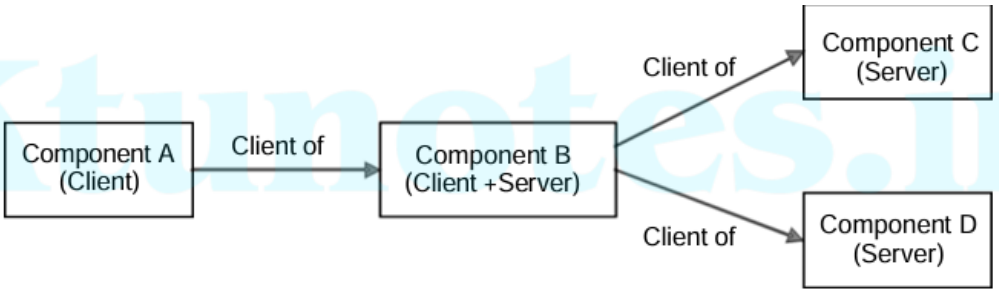


Fig. 9.7: Acting as a Client and a Server

Ktunotes.in

4. Electronic Data Interchange (EDI)

Electronic Data Interchange (EDI) is a system that enables the **direct transfer of structured data between computers**. The main objective of EDI is to **eliminate paper-based documents** and manual handling of business transactions.

In EDI, business documents such as **purchase orders, invoices, delivery notes, receiving advice, and payment records** are transmitted in a **standardized electronic format** rather than as printed documents. These electronic documents are exchanged over secure communication networks that connect trading partners.

Unlike electronic mail, which transmits **unstructured text messages**, EDI transmits **actual business transactions** with predefined fields such as transaction date, transaction amount, sender details, and receiver details. This structured format allows the data to be processed automatically by the receiving system without re-entering information.

EDI significantly **reduces operational costs** by eliminating paper handling, printing, and manual data entry. It also **saves time and manpower**, as electronic transmission is much faster than postal mail or manual delivery. Since data is transferred directly between systems, the chances of errors caused by re-keying data are greatly reduced.

Thus, EDI is a transformational client/server technology that improves **speed, accuracy, efficiency, and reliability** in inter-organizational business communication

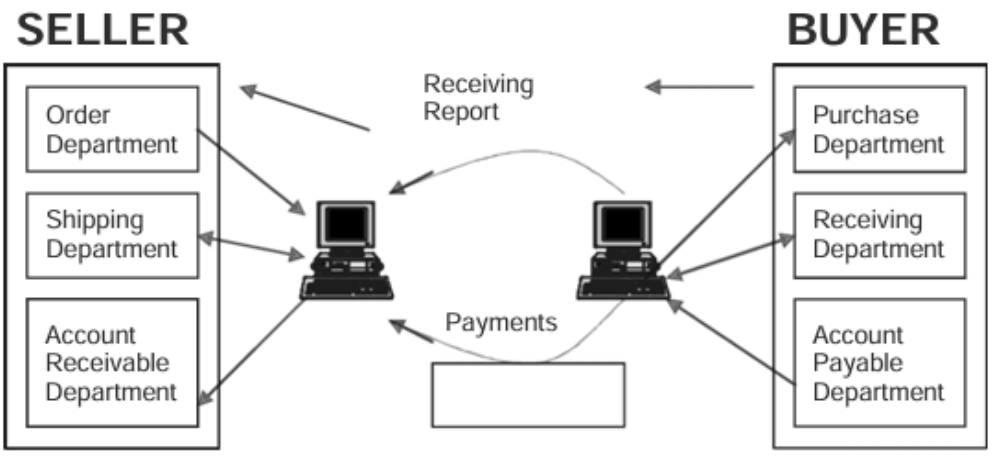


Fig. 9.9: EDI as an Example

5. Geographic Information System (GIS)

A Geographic Information System (GIS) is a system that provides the capability to view, store, analyze, and manage geographic or spatial information related to real-world features such as roads, sewers, electrical cables, soil composition, mineral resources, and water sources. GIS becomes truly useful when it is integrated with organizational business processes and databases.

In GIS, geographic information is stored as a set of layers. The base layer may contain features such as street layouts, while additional thematic layers may represent wetlands, underground water sources, or utility networks. Users can generate customized maps by

selecting and combining relevant layers according to their needs. For example, selecting street and wetland layers together shows the relationship between roads and wetlands.

Each feature on a GIS map is fixed in space using spatial data, which defines the exact position of objects using numerical coordinates in three-dimensional space. Along with spatial data, GIS allows attribute data to be associated with map objects. Attribute data may include descriptive information, measurements, dates, legal details, or comments, and is usually stored in relational database management systems (RDBMS).

GIS applications are naturally suited to client/server architecture. Powerful client workstations handle map visualization and interaction, while servers manage shared geographic layers and related databases. This allows multiple departments to access and update data concurrently. When GIS data is combined with other organizational databases, it provides higher value—for example, combining voter lists with street maps helps in planning polling booth locations.

Thus, GIS is a transformational system that uses client/server technology to integrate spatial data with business data, enabling better planning, decision-making, and resource management

6. Electronic Document Management (EDM) – Multimedia

Electronic Document Management refers to the computer-based management and manipulation of information that exists in different forms such as forms, blueprints, x-rays, microfilm, fingerprints, photographs, and handwritten or typed documents. When multimedia is included, it also supports audio and video information.

EDM uses an electronic folder-based interface, which is more flexible and intuitive than traditional filing cabinets. Documents stored electronically are always available on the desktop, can be accessed by multiple users simultaneously, and are automatically stored in the correct location. This eliminates physical handling and loss of documents.

These systems are especially important in large organizations and government services where billions of documents must be stored and accessed daily. EDM enables rapid filing, retrieval, sharing, and secure access to documents, ensuring that information is available only to users with a “need and right to know”.

Electronic document management systems rely on client/server technology, where servers store and manage large multimedia repositories and client workstations provide user-friendly access interfaces. Although traditionally expensive, the notes mention that costs are decreasing as computer technology advances, making EDM increasingly practical.

Thus, electronic document management is a transformational system that improves efficiency, accessibility, security, and collaboration in handling large volumes of organizational information

7. Full-Text Retrieval

Full-text retrieval is an area of explosive growth, made practical by the availability of high-powered workstations and RISC servers. Originally developed for military applications such as scanning covert transmissions and media communications, it has now become a mainstream technology used in business and government. In full-text retrieval systems, the entire content of documents is indexed and searched, allowing users to perform multiple-criteria searches over extremely large volumes of text in a very short time. For example, major news services provide electronic feeds that are continuously scanned to identify significant events and trends, and systems such as the Dow Jones news retrieval service provide access to 10^{12} bytes of textual data, with searches completed in seconds.

The key technologies that make full-text retrieval viable include Optical Character Recognition (OCR) and Intelligent Character Recognition (ICR), which convert printed and handwritten paper documents into machine-readable text files. Additional enabling technologies include optical storage, large D-RAM, powerful processors, software indexing algorithms, and high-resolution monitors. Full-text indexing is a CPU-intensive operation, but the availability of low-cost, high-performance hardware has made such systems practical. Products such as Lotus Magellan allow users to create full-text indexes of files and access documents in their native formats, while advanced indexing supports Boolean search operations for high-performance retrieval. Thus, full-text retrieval is a transformational system t**hat enables rapid and efficient access to massive collections of textual information**